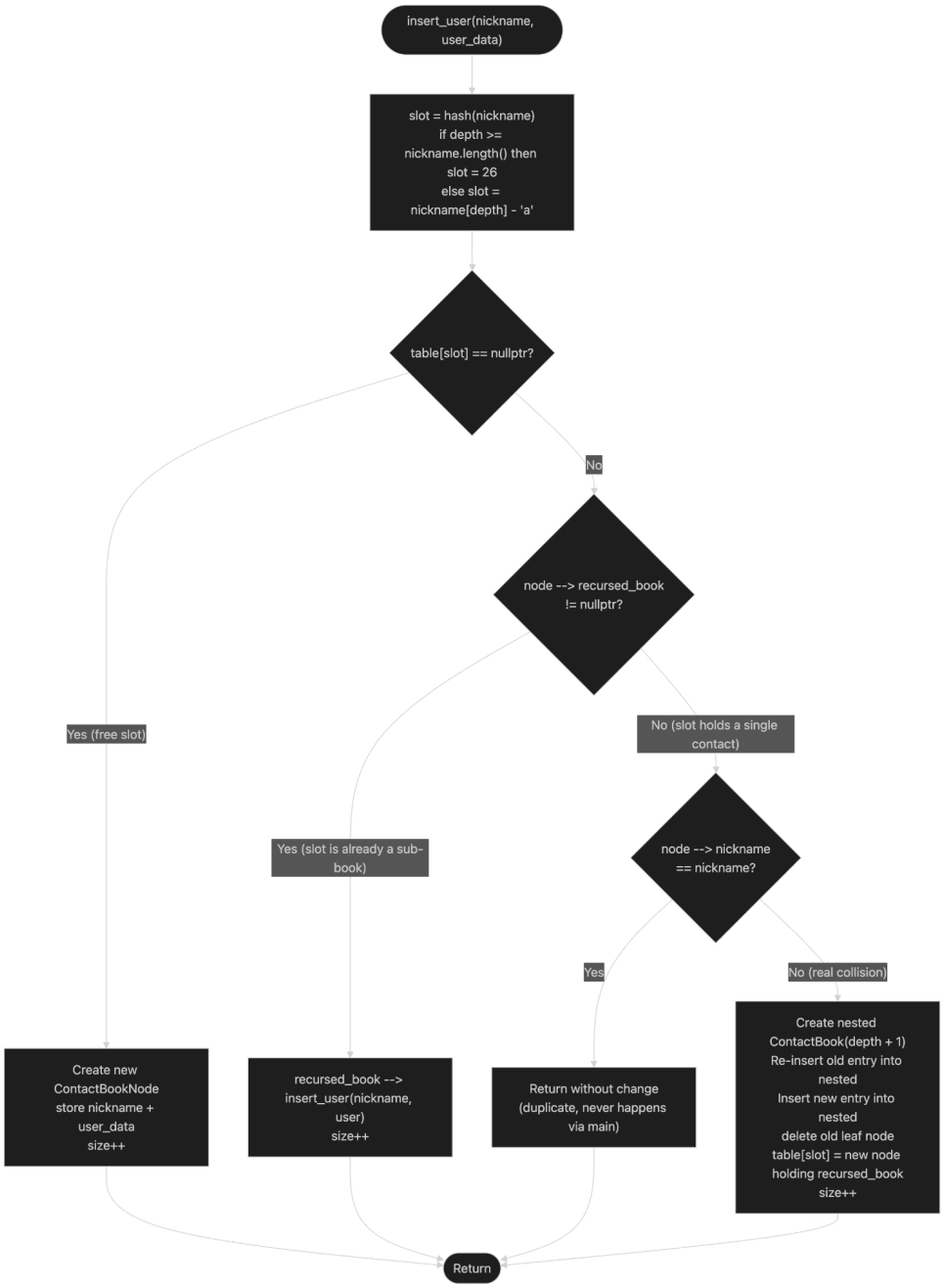
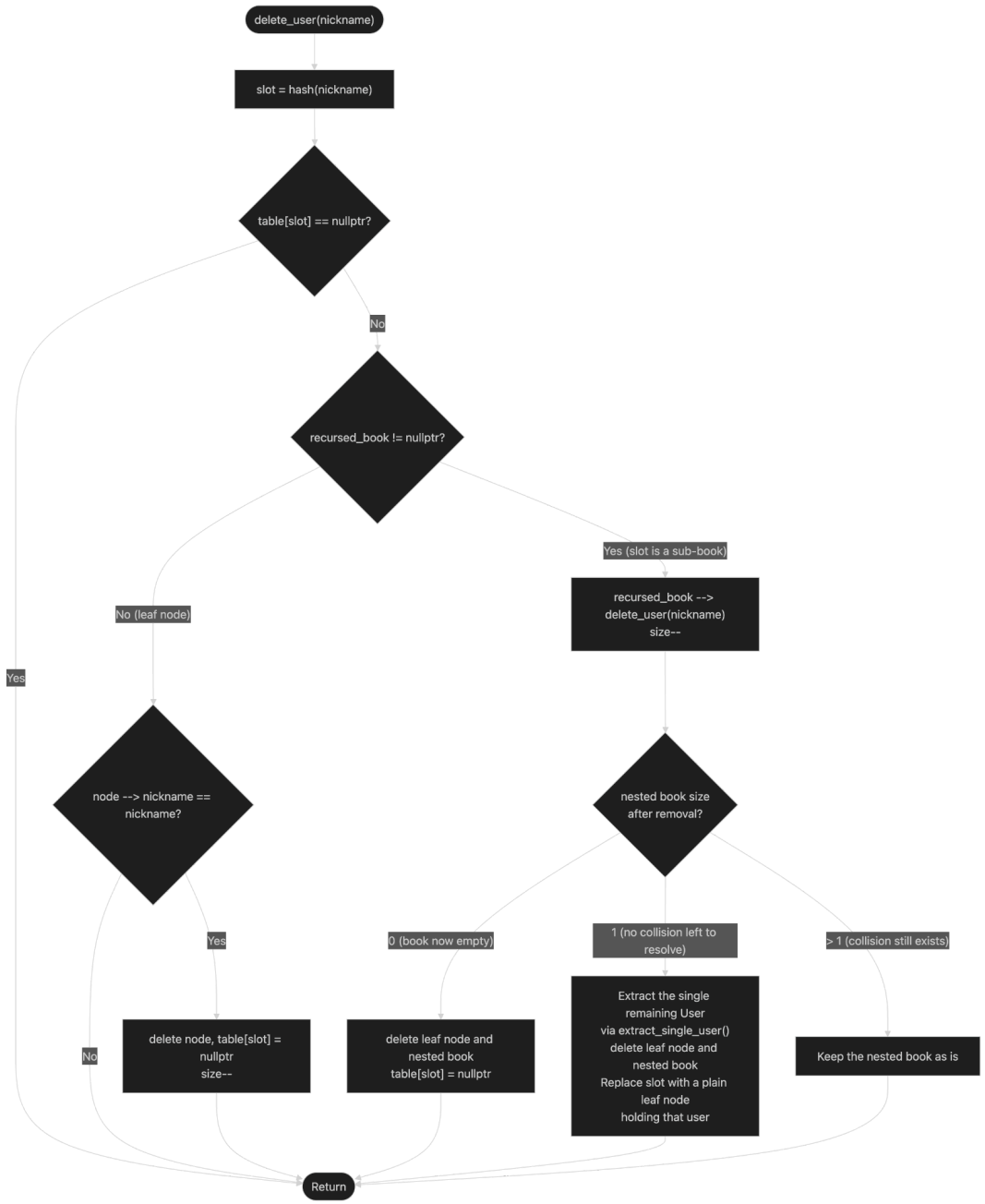
Task 1: insert_user() — Collision Triggers a New Hash Table

Explanation: The hash at depth d is simply the d-th (0-indexed) character mapped to 0.25; a nickname that has run out of characters lands in slot 26. A collision never produces a linked list. Instead, the existing leaf entry and the new entry are both pushed one level deeper into a fresh ContactBook whose hash uses the *next* character. Because all nicknames have finite length, the recursion is guaranteed to terminate two distinct nicknames must eventually differ (or one ends, hitting slot 26). Each level's size++ keeps the count maintained on every book along the path (Task 2).



Task 3: delete_user() — Removal with Collapse of Unnecessary Books

Explanation: After a removal, the recursion unwinds and each ancestor book checks whether the nested book it created is still justified. If the nested book is empty, the whole slot is cleared. If it contains exactly one user, the collision that justified creating it is gone, so the sub-book is collapsed back into a single leaf node (per the Task 3 requirement: non-root books with only a single user are replaced by that entry). extract_single_user() walks down through any further nesting to retrieve that one remaining user, cleaning up empty sub-books on its way back up.



Task 4: search() — Prefix Query

Explanation: The search walks the same character-by-character path an insert would. Once the query prefix is exhausted (its length no longer exceeds the current depth), *every* contact in that sub-book necessarily has the prefix, so the answer is simply size > 0. At a leaf node, an explicit substring comparison confirms the stored nickname actually starts with the query (important because slot 26 holds shorter nicknames whose hash came from the empty character). This works because entries are effectively ordered by nickname along the hash path which is exactly why the structure resembles a trie (prefix tree), the answer to Task 5: each depth level branches on the next character, 26 acts as the end-of-string marker, and prefix lookups traverse only one path plus everything beneath it.

